

中文分词模拟器

题目描述

给定一个连续不包含空格的 **字符串**，该字符串仅包含英文小写字母及英文标点符号（逗号、分号、句号），同时给定词库，对该字符串进行精确分词。

说明：

1. 精确分词：字符串分词后，不会出现重叠。即"ilovechina"，不同词库可分割为"i,love,china"，"ilove,china"，不能分割出现重叠的"i,ilove,china"，i 出现重叠
2. 标点符号不成词，仅用于断句
3. 词库：根据外部知识库统计出来的常用词汇例：dictionary = ["i", "love", "china", "lovechina", "ilove"]
4. 分词原则：采用分词顺序优先且最长匹配原则

"ilovechina"，假设分词结果 [i,ilove,lo,love,ch,china,lovechina]，则输出 [ilove,china]

错误输出：[i,lovechina]，原因："ilove" > 优先于 "lovechina" 成词

错误输出：[i,love,china]，原因："ilove" > "i"遵循最长匹配原则

输入描述

第一行输入待 **分词** 语句 "ilovechina"

- 字符串长度限制：0 < length < 256

第二行输入中文词库 "i,love,china,ch,na,ve,lo,this,is,this,word"

- 词库长度限制：1 < length < 100000

输出描述

按顺序输出分词结果 "i,love,china"

用例

输入	ilovechina i,love,china,ch,na,ve,lo,this,is,the,word
输出	i,love,china
说明	无

输入	iat i,love,china,ch,na,ve,lo,this,is,the,word,beauti,tiful,ful
输出	i,a,t
说明	单个字母, 不在词库中且不成词则输出单个字母

输入	ilovechina,thewordisbeautiful i,love,china,ch,na,ve,lo,this,is,the,word,beauti,tiful,ful
输出	i,love,china,the,word,is,beauti,ful
说明	标点符号为英文标点符号

题目解析

本题感觉题意不是很清晰。

我对题目的理解是：

句子：ilovechina

词库：[i,love,china,ch,na,ve,lo,this,is,the,word]

现在要用词库里面的词汇组成句子。并且选择词汇时，优先选择词库中靠前的，且长度最长的。

比如组成句子“ilovechina”的第一个词汇，必然是 "i" 开头的，因此我们去词库中找 "i" 开头的词汇，按词库顺序依次是：

- i
- is

其中 is 虽然是 i 开头，但是不符合句子后续部分要求，因此选择词库中词汇 “i”。

此时句子还剩下 "lovechina" 没有分词，则继续在词库中查找 "l" 开头的词汇，按词库顺序依次是：

- love
- lo

其中 "love" 是顺序靠前，且长度较长的，因此选择词库中词汇 "love"。

此时句子还剩下 "china" 没有分词，则继续在词库中查找 "c" 开头的词汇，按词库顺序依次是：

- china
- ch

其中 "china" 是顺序靠前，且长度较长的，因此选择词库中词汇 "china"。

此时句子"ilovechina" 完成分词，分词结果为：["i", "love", "china"]。

本题，我的疑惑主要在于用例2：



句子: "iat"

词库: [i,love,china,ch,na,ve,lo,this,is,the,word,beauti,tiful,ful]

按照之前的逻辑, 首先找到组成句子的第一个词汇, 该词汇必然以"i"开头, 则匹配到词库中的词汇"i"。

接下来句子还剩"at", 再去词库中查找"a"开头的词汇, 发现没有。那么此时我的疑问来了:

- 是只针对句子剩余部分"at", 进行输出单个字母的操作
- 还是针对整个句子"iat", 进行输出单个字母的操作

对于这个用例, 两种策略的输出结果都是一样, 因为这个用例给的太有歧义了。

比如我给个例子:

句子: iloveat

词库: [i,love,china,ch,na,ve,lo,this,is,the,word,beauti,tiful,ful]

此时输出: i,love,a,t

还是输出: i,l,o,v,e,a,t

不在词库中且不成词则输出单个字母

上面我给例子, i 在词库中, love在词库中, at不在词库中, 因此只针对at执行输出单个字母操作。

一次你, 我理解是输出: i,love,a,t

另外, 不成词是什么意思呢?

题目中关于"不成词", 只有一个相关描述: 标点符号不成词, 仅用于断句

那么是不是就是说, 标点符号不成词, 英文小写字母成词呢? 我理解应该是的。

JS算法源码

运行

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  // 输入输出处理
6  void (async function () {
7      const sentences = (await readline()).split(/[,.;]/).filter((a) => a !== "");
8      const words = (await readline()).split(/[,.;]/).filter((a) => a !== "");
9
10     console.log(getResult(sentences, words));
11 })();
12
13 function getResult(sentences, words) {
14     // wordSet 记录词库词汇
15     const wordSet = new Set(words);
16
17     // ans记录最终分词结果
18     const ans = [];
19
20     while (sentences.length > 0) {
21         const sentence = sentences.shift();
22
23         let r = sentence.length;
24         for (; r > 0; r--) {
25             // 截取句子 [0,r) 范围子串词汇，这样的就能实现优先最长匹配，并且由于是必须从0索引开始截取，因此满足了分词顺序优先
26             const fragment = sentence.slice(0, r);
27
28             // 若词库中是否存在该子串词汇
29             if (wordSet.has(fragment)) {
30                 // 则将对应子串词汇纳入结果
31                 ans.push(fragment);
32                 wordSet.delete(fragment); // 我理解词库中每个词汇只能使用一次，因此这里将词库中使用过的词汇移除
33
34                 // 若子串词汇只是句子部分，则句子剩余部分还要继续去词库中查找
35                 if (r < sentence.length) {
```

```

36         sentences.unshift(sentence.slice(r));
37     }

38
39     break;
40 }
41 }
42
43 // 没有在词库中找到对应子串词汇，则输出单个字母
44 if (r == 0) {
45     // 注意，这里只放一个字母到结果中，句子剩余部分继续去词库中查找
46     ans.push(sentence[0]);
47
48     if (sentence.length > 1) {
49         sentences.unshift(sentence.slice(1));
50     }
51 }
52 }
53
54 return ans.join(",");
55 }

```

Java算法源码

```

1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         String[] sentences = sc.nextLine().split("[,.;]");
8         String[] words = sc.nextLine().split("[,.;]");
9
10        System.out.println(getResult(sentences, words));
11    }

```

```

12 |
13 |     public static String getResult(String[] sentences, String[] words) {
14 |         // wordSet 记录词库词汇
15 |         HashSet<String> wordSet = new HashSet<>(Arrays.asList(words));
16 |
17 |         // queue记录待分词语句
18 |         LinkedList<String> queue = new LinkedList<>(Arrays.asList(sentences));
19 |
20 |         // ans记录最终分词结果
21 |         LinkedList<String> ans = new LinkedList<>();
22 |
23 |         while (queue.size() > 0) {
24 |             // 待分词的句子
25 |             String sentence = queue.removeFirst();
26 |
27 |             int r = sentence.length();
28 |             for (; r > 0; r--) {
29 |                 // 截取句子 [0,r) 范围子串词汇, 这样的就能实现优先最长匹配, 并且由于是必须从0索引开始截取, 因此满足了分词顺序优先
30 |                 String fragment = sentence.substring(0, r);
31 |
32 |                 // 若词库中是否存在该子串词汇
33 |                 if (wordSet.contains(fragment)) {
34 |                     // 则将对应子串词汇纳入结果
35 |                     ans.addLast(fragment);
36 |                     wordSet.remove(fragment); // 我理解词库中每个词汇只能使用一次, 因此这里将词库中使用过的词汇移除
37 |
38 |                     // 若子串词汇只是句子部分, 则句子剩余部分还要继续去词库中查找
39 |                     if (r < sentence.length()) {
40 |                         queue.addFirst(sentence.substring(r));
41 |                     }
42 |
43 |                     break;
44 |                 }
45 |             }
46 |
47 |             // 没有在词库中找到对应子串词汇, 则输出单个字母
48 |             if (r == 0) {

```

```

49 // 注意，这里只放一个字母到结果中，句子剩余部分继续去词库中查找
50
51
52     if (sentence.length() > 1) {
53         queue.addFirst(sentence.substring(1));
54     }
55 }
56 }
57
58 StringJoiner sj = new StringJoiner(",");
59 ans.forEach(sj::add);
60
61 return sj.toString();
62 }
63 }

```

```
ans.add(sentence.charAt(0) + "");
```

Python算法源码

```

1 import re
2
3 # 输入获取
4 sentences = list(filter(lambda x: x != "", re.split(r"[.,;]", input())))
5 words = list(filter(lambda x: x != "", re.split(r"[.,;]", input())))
6
7
8 def getResult():
9     # wordSet 记录词库词汇
10    wordSet = set(words)
11    # ans记录最终分词结果
12    ans = []
13
14    while len(sentences) > 0:
15        # 待分词的句子
16        sentence = sentences.pop(0)
17

```



```

18     r = len(sentence)19     while r > 0:
20         # 截取句子 [0,r) 范围子串词汇，这样的就能实现优先最长匹配，并且由于是必须从0索引开始截取，因此满足了分词顺序优先
21         fragment = sentence[0:r]
22
23         # 若词库中是否存在该子串词汇
24         if fragment in wordSet:
25             # 则将对应子串词汇纳入结果
26             ans.append(fragment)
27             wordSet.remove(fragment) # 我理解词库中每个词汇只能使用一次，因此这里将词库中使用过的词汇移除
28
29             # 若子串词汇只是句子部分，则句子剩余部分还要继续去词库中查找
30             if r < len(sentence):
31                 sentences.insert(0, sentence[r:])
32
33             break
34
35         r -= 1
36
37     # 没有在词库中找到对应子串词汇，则输出单个字母
38     if r == 0:
39         # 注意，这里只放一个字母到结果中，句子剩余部分继续去词库中查找
40         ans.append(sentence[0])
41
42         if len(sentence) > 1:
43             sentences.insert(0, sentence[1:])
44
45     return ",".join(ans)
46
47
48 print(getResult())

```

C算法源码

```

1 #include <stdio.h>
2 #include <stdlib.h>

```

```
3  #include <string.h>
4
5  #define MAX_LENGTH 256
6  #define MAX_WORDS_LENGTH 100000
7  #define delimiter ",.;"
8  #define TRUE 1
9  #define FALSE 0
10
11 /** 实现双向链表 **/
12 typedef struct ListNode {
13     char *val;
14     struct ListNode *prev;
15     struct ListNode *next;
16 } ListNode;
17
18 ListNode *new_ListNode(char *val) {
19     ListNode *node = (ListNode *) malloc(sizeof(ListNode));
20     node->val = (char *) calloc(MAX_LENGTH, sizeof(char));
21     strcpy(node->val, val);
22     node->prev = NULL;
23     node->next = NULL;
24     return node;
25 }
26
27 typedef struct LinkedList {
28     int size;
29     ListNode *head;
30     ListNode *tail;
31 } LinkedList;
32
33 LinkedList *new_LinkedList() {
34     LinkedList *link = (LinkedList *) malloc(sizeof(LinkedList));
35     link->size = 0;
36     link->head = NULL;
37     link->tail = NULL;
38     return link;
39 }
```

```
40
41 | void addFirst_LinkedList(LinkedList *link, char *val) {
42     ListNode *node = new_ListNode(val);
43
44     if (link->size == 0) {
45         link->head = node;
46         link->tail = node;
47     } else {
48         node->next = link->head;
49         link->head->prev = node;
50         link->head = node;
51     }
52
53     link->size++;
54 }
55
56 void addLast_LinkedList(LinkedList *link, char *val) {
57     ListNode *node = new_ListNode(val);
58
59     if (link->size == 0) {
60         link->head = node;
61         link->tail = node;
62     } else {
63         link->tail->next = node;
64         node->prev = link->tail;
65         link->tail = node;
66     }
67
68     link->size++;
69 }
70
71 char *removeFirst_LinkedList(LinkedList *link) {
72     if (link->size == 0) exit(-1);
73
74     ListNode *removed = link->head;
75
76     if (link->size == 1) {
```

```

77         link->head = NULL; 78         link->tail = NULL;
79     } else {
80         link->head = link->head->next;
81         link->head->prev = NULL;
82     }
83
84     link->size--;
85
86     char *res = (char *) calloc(MAX_LENGTH, sizeof(char));
87     strcpy(res, removed->val);
88
89     free(removed);
90
91     return res;
92 }
93
94 char *removeLast_LinkedList(LinkedList *link) {
95     if (link->size == 0) exit(-1);
96
97     ListNode *removed = link->tail;
98
99     if (link->size == 1) {
100         link->head = NULL;
101         link->tail = NULL;
102     } else {
103         link->tail = link->tail->prev;
104         link->tail->next = NULL;
105     }
106
107     link->size--;
108
109     char *res = (char *) calloc(MAX_LENGTH, sizeof(char));
110     strcpy(res, removed->val);
111
112     free(removed);
113
114     return res;

```

```
115 | }
    | 116 |
117 | /** 实现字符串截取 **/
118 | char *substr(char *s, int l, int r) {
119 |     int len = r - l;
120 |     char *res = (char *) calloc(len + 1, sizeof(char));
121 |     strncpy(res, s + l, len);
122 |     return res;
123 | }
124 |
125 | int main() {
126 |     char s1[MAX_LENGTH];
127 |     gets(s1);
128 |
129 |     // sentences记录待分词语句
130 |     LinkedList *sentences = new_LinkedList();
131 |     char *token1 = strtok(s1, delimiter);
132 |     while (token1 != NULL) {
133 |         addLast_LinkedList(sentences, token1);
134 |         token1 = strtok(NULL, delimiter);
135 |     }
136 |
137 |     char s2[MAX_WORDS_LENGTH];
138 |     gets(s2);
139 |
140 |     // words 记录词库词汇
141 |     LinkedList *words = new_LinkedList();
142 |     char *token2 = strtok(s2, delimiter);
143 |     while (token2 != NULL) {
144 |         addLast_LinkedList(words, token2);
145 |         token2 = strtok(NULL, delimiter);
146 |     }
147 |
148 |     // ans记录最终分词结果
149 |     LinkedList *ans = new_LinkedList();
150 |
151 |     while (sentences->size > 0) {
```

```

152 char *sentence = removeFirst_LinkedList(sentences);
153
154 int len = (int) strlen(sentence);
155 int r = len;
156
157 for (; r > 0; r--) {
158     // 截取句子 [0,r) 范围子串词汇，这样的就能实现优先最长匹配，并且由于是必须从0索引开始截取，因此满足了分词顺序优先
159     char *fragment = substr(sentence, 0, r);
160
161     int find = FALSE;
162
163     ListNode *cur = words->head;
164     while (cur != NULL) {
165         // 若词库中是否存在该子串词汇
166         if (strcmp(cur->val, fragment) == 0) {
167             // 则将对应子串词汇纳入结果
168             addLast_LinkedList(ans, fragment);
169             strcpy(cur->val, ""); // 我理解词库中每个词汇只能使用一次，因此这里将词库中使用过的词汇移除
170
171             // 若子串词汇只是句子部分，则句子剩余部分还要继续去词库中查找
172             if (r < len) {
173                 addFirst_LinkedList(sentences, substr(sentence, r, len));
174             }
175
176             find = TRUE;
177
178             break;
179         }
180         cur = cur->next;
181     }
182
183     if (find) {
184         break;
185     }
186 }
187
188 // 没有在词库中找到对应子串词汇，则输出单个字母

```

```

189         if (r == 0) {190 |           // 注意，这里只放一个字母到结果中，句子剩余部分继续去词库中查找
191             char tmp[] = {sentence[0], '\0'};
192             addLast_LinkedList(ans, tmp);
193
194             if (len > 1) {
195                 addFirst_LinkedList(sentences, substr(sentence, 1, len));
196             }
197         }
198     }
199
200     ListNode *cur = ans->head;
201
202     while (cur != NULL) {
203         printf("%s", cur->val);
204         cur = cur->next;
205         if (cur != NULL) {
206             printf(",");
207         }
208     }
209
210     return 0;
211 }

```

C++ 算法源码

```

1  #include<bits/stdc++.h>
2  using namespace std;
3
4  vector<string> splitCin(set<char> &separator) {
5      string s;
6      getline(cin, s);
7
8      string tmp;
9      vector<string> res;
10

```

```
11     for (const auto &c: s) {
12         if (separator.count(c) > 0) {
13             res.emplace_back(tmp);
14             tmp.clear();
15         } else {
16             tmp += c;
17         }
18     }
19
20     if (!tmp.empty()) {
21         res.emplace_back(tmp);
22     }
23
24     return res;
25 }
26
27 int main() {
28     set<char> separator({' ', '.', ';'});
29
30     vector<string> sentences = splitCin(separator);
31     vector<string> words = splitCin(separator);
32
33     // wordSet 记录词库词汇
34     set<string> wordSet;
35     for (const auto &word: words) {
36         wordSet.insert(word);
37     }
38
39     // queue记录待分词语句
40     deque<string> queue;
41     for (const auto &sentence: sentences) {
42         queue.emplace_back(sentence);
43     }
44
45     // ans记录最终分词结果
46     deque<string> ans;
47
```



```

48 while (!queue.empty()) {
49     // 待分词的句子
50     string sentence = queue.front();
51     queue.pop_front();
52
53     int r = sentence.length();
54     for (; r > 0; r--) {
55         // 截取句子 [0,r) 范围子串词汇, 这样的就能实现优先最长匹配, 并且由于是从0索引开始截取, 因此满足了分词顺序优先
56         string fragment = sentence.substr(0, r);
57
58         // 若词库中是否存在该子串词汇
59         if (wordSet.count(fragment) > 0) {
60             // 则将对子串词汇纳入结果
61             ans.emplace_back(fragment);
62             wordSet.erase(fragment); // 我理解词库中每个词汇只能使用一次, 因此这里将词库中使用过的词汇移除
63
64             // 若子串词汇只是句子部分, 则句子剩余部分还要继续去词库中查找
65             if (r < sentence.length()) {
66                 queue.push_front(sentence.substr(r));
67             }
68
69             break;
70         }
71     }
72
73     // 没有在词库中找到对应子串词汇, 则输出单个字母
74     if (r == 0) {
75         // 注意, 这里只放一个字母到结果中, 句子剩余部分继续去词库中查找
76         ans.emplace_back(1, sentence[0]);
77
78         if (sentence.length() > 1) {
79             queue.push_front(sentence.substr(1));
80         }
81     }
82 }
83
84 // 打印结果

```

```
85 | cout << ans[0];86 |     for (int i = 1; i < ans.size(); i++) {
87 |         cout << "," << ans[i];
88 |     }
89 |     cout << endl;
90 |
91 |     return 0;
92 | }
```